DeObfusca-AI: GNN Deobfuscator Architecture & Mathematical Documentation

1. Executive Summary

This document provides a comprehensive technical analysis of the Graph Neural Network (GNN) based deobfuscation system implemented in `train_gnn.py`. The system is designed to detect "junk" instructions injected by obfuscators (specifically OLLVM-style control flow flattening and bogus control flow) within x86 assembly code.

The architecture leverages a **Graph Transformer** model, which combines the sequential nature of assembly instructions (via Positional Encodings) with the structural dependencies of the Control Flow Graph (CFG) (via Edge-Augmented Attention).

---

2. Mathematical Foundations of Deep Learning Layers

This section details the mathematical operations governing the layers used in the `GNN_Deobfuscator`.

2.1. Embedding Layers

Concept: Neural networks cannot process discrete strings (like "MOV", "EAX"). Embeddings map these discrete tokens into a continuous vector space where semantic relationships can be learned.

Mathematical Formulation: Let V be the vocabulary size and d_{model} be the embedding dimension. The embedding layer is a learnable matrix $W_E \in \mathbb{R}^{V \times d_{model}}$.

For a given token index $t \in \{0, ..., V-1\}$, the embedding operation is a lookup:

$$\mathbf{e}_t = \text{Embedding}(t) = W_E[t, :]$$

In our model, we have three distinct embedding spaces: 1. **Mnemonic Embedding** (E_{mnem}): Captures the semantics of the opcode (e.g., `ADD` vs `JMP`). 2. **Operand Type Embedding** (E_{op}): Captures the nature of operands (Register, Memory, Immediate). 3. **Edge Attribute Embedding** (E_{edge}): Captures the type of control flow edge (Sequential, Jump, Branch).

2.2. Feature Fusion

Concept: To create a single vector representation for an assembly instruction node, we must combine the mnemonic embedding with its operand embeddings.

Mathematical Formulation: Given input vectors $\mathbf{x}_{mnem} \in \mathbb{R}^d$, $\mathbf{x}_{op1} \in \mathbb{R}^{d_{op}}$, and $\mathbf{x}_{op2} \in \mathbb{R}^{d_{op}}$.

We first concatenate them:

$$\mathbf{x}_{raw} = [\mathbf{x}_{mnem} \parallel \mathbf{x}_{op1} \parallel \mathbf{x}_{op2}] \in \mathbb{R}^{d+2d_{op}}$$

Then, we project them back to the model dimension d_{model} using a linear transformation (Fusion Layer):

$$\mathbf{h}_0 = \mathbf{x}_{raw}\mathbf{W}_{fusion} + \mathbf{b}_{fusion}$$

Where $\mathbf{W}_{fusion} \in \mathbb{R}^{(d+2d_{op})\times d_{model}}$.

2.3. Positional Encoding (Sinusoidal)

Concept: Graph Neural Networks are typically permutation invariant. However, assembly code has a strict sequential execution order (memory address order). To inject this sequence information, we add positional encodings to the node features.

Mathematical Formulation: For a position pos and dimension index $2i$ or $2i+1$:

$$PE(pos, 2i) = \sin\left(\frac{pos}{10000^{2i/d_{model}}}\right)$$

$$PE(pos, 2i+1) = \cos\left(\frac{pos}{10000^{2i/d_{model}}}\right)$$

The final input to the first transformer layer is:

$$\mathbf{h}_{input} = \mathbf{h}_0 + PE$$

2.4. Edge-Augmented Multi-Head Attention

Concept: Standard Transformers calculate attention based solely on node similarity. In a Control Flow Graph (CFG), the *type* of edge connecting two nodes (e.g., a conditional jump vs. a fall-through) is critical context. We inject this structural information directly into the attention mechanism.

Mathematical Formulation:

Let $H \in \mathbb{R}^{N\times d_{model}}$ be the input node features. We compute Query (Q), Key (K), and Value (V) matrices for each head h:

$$Q_h = HW_Q^h, \quad K_h = HW_K^h, \quad V_h = HW_V^h$$

Edge Bias Injection: Let $E_{attr} \in \mathbb{R}^{M \times d_{edge}}$ be the edge features for M edges. We project these to the number of heads:

$$E_{bias} = E_{attr} W_{edge} \in \mathbb{R}^{M \times H_{heads}}$$

The attention score A_{ij} between node i and node j connected by edge e_{ij} is:

$$\text{Score}_{ij}^{h} = \frac{(Q_h)_i (K_h)_j^T}{\sqrt{d_k}} + (E_{bias})_{e_{ij},h}$$

The attention weights are obtained via Softmax:

$$\alpha_{ij}^{h} = \text{Softmax}_j(\text{Score}_{ij}^{h})$$

The output of head h is the weighted sum of values:

$$\text{Head}_h = \sum_{j \in \mathcal{N}(i)} \alpha_{ij}^{h} (V_h)_j$$

Finally, all heads are concatenated and projected:

$$\text{MultiHead}(H) = [\text{Head}_1 \parallel ... \parallel \text{Head}_k] W_O$$

2.5. Layer Normalization

Concept: LayerNorm normalizes the inputs across the feature dimension for each sample independently, stabilizing the gradients.

Mathematical Formulation: For a vector $\mathbf{x}$:

$$\hat{\mathbf{x}} = \frac{\mathbf{x} - \mu}{\sqrt{\sigma^2 + \epsilon}}$$

$$\text{LayerNorm}(\mathbf{x}) = \gamma \hat{\mathbf{x}} + \beta$$

Where μ and σ^2 are the mean and variance of elements in $\mathbf{x}$, and γ, β are learnable parameters.

2.6. Feed-Forward Network (FFN)

Concept: The FFN provides point-wise non-linearity, allowing the model to process information at each node individually after aggregating context via attention.

Mathematical Formulation:

$$\text{FFN}(\mathbf{x}) = \text{Dropout}(\text{GELU}(\mathbf{x}W_1 + b_1))W_2 + b_2$$

We use the **GELU** (Gaussian Error Linear Unit) activation function:

$$\text{GELU}(x) = x\Phi(x) \approx 0.5x(1 + \tanh(\sqrt{2/\pi}(x + 0.044715x^3)))$$

2.7. Focal Loss

Concept: In deobfuscation, "Real" instructions vastly outnumber "Junk" instructions. Standard Cross-Entropy loss would be overwhelmed by the easy "Real" examples. Focal Loss down-weights easy examples and focuses training on hard misclassifications.

Mathematical Formulation: Let p_t be the model's estimated probability for the true class.

Standard Cross Entropy:

$$CE(p_t) = -\log(p_t)$$

Focal Loss:

$$FL(p_t) = -\alpha_t(1 - p_t)^\gamma \log(p_t)$$

- $(1 - p_t)^\gamma$ is the **modulating factor**. If $p_t \approx 1$ (easy example), the factor approaches 0, reducing loss.
- α_t is the **balancing factor** for class weights.

---

3. Codebase Analysis & Implementation

This section dissects `train_gnn.py` block by block.

3.1. Configuration (`HPARAMS`)

This dictionary controls the model architecture and training dynamics.

```
HPARAMS = {
    'embed_dim': 256,       # d_model: Size of internal vectors
    'op_embed_dim': 32,     # Dimension for operand types
    'layers': 6,            # Number of Transformer blocks
    'heads': 8,             # Number of attention heads
    'lr': 5e-5,             # Learning Rate
```

```
    'batch_size': 8,        # Graphs per batch
    'epochs': 20,           # Training duration
    'obfuscate_p': 0.3,     # Probability of injecting junk into a sample
    'gamma': 2.0,           # Focal Loss focusing parameter
    'alpha': 0.75,          # Focal Loss balance parameter
    'target_files': 1200    # Dataset size target
}
```

3.2. Vocabulary (`Vocab`)

The `Vocab` class handles the mapping from assembly mnemonics to integer IDs.

```
class Vocab:
    """Maps x86 assembly mnemonics to integer IDs."""
    TOKENS = [
        '<PAD>', '<UNK>', '<S>', '</S>',
        'MOV', 'PUSH', 'POP', 'LEA', 'NOP', 'XCHG', 'IN', 'OUT',
        # ... Arithmetic, Logic, Control Flow instructions ...
        'LDR', 'STR', 'BL', 'BX', 'SVC', 'COPY', 'LOAD', 'STORE', 'BRANCH'
    ]

    def __init__(self):
        self.map = {t: i for i, t in enumerate(self.TOKENS)}
        self.unk = self.map['<UNK>']

    def get(self, token):
        return self.map.get(token.upper().strip(), self.unk)
```

Analysis: - **Special Tokens**: `<PAD>` (0) for batching, `<UNK>` (1) for unseen instructions. - **Normalization**: `token.upper().strip()` ensures case-insensitivity.

3.3. The Loss Function (`FocalLoss`)

Implementation of the formula $FL(p_t) = -\alpha(1-p_t)^\gamma \log(p_t)$.

```
class FocalLoss(nn.Module):
    def __init__(self, alpha=0.75, gamma=2.0):
        super().__init__()
        self.alpha = alpha
        self.gamma = gamma

    def forward(self, inputs, targets):
        # 1. Compute standard BCE Loss (raw logits -> loss)
        bce_loss = F.binary_cross_entropy_with_logits(inputs, targets, reduction='none')

        # 2. Calculate p_t (probability of true class)
        # Since BCE = -log(p_t), then p_t = exp(-BCE)
```

```
        pt = torch.exp(-bce_loss)

        # 3. Apply Focal Loss formula
        focal_loss = self.alpha * (1 - pt) ** self.gamma * bce_loss

        return focal_loss.mean()
```

3.4. Edge-Augmented Attention (`EdgeAugmentedAttention`)

This is the custom attention mechanism that makes the model "Graph-Aware".

```
class EdgeAugmentedAttention(nn.Module):
    def __init__(self, dim, heads, edge_dim, drop=0.1):
        super().__init__()
        # ... Initialization of Linear layers Q, K, V ...
        self.edge_proj = nn.Linear(edge_dim, heads, bias=False)

    def forward(self, x, edge_index, edge_attr):
        r, c = edge_index  # Source and Target node indices

        # Linear Projections & Reshaping for Multi-Head
        q = self.q(x).view(-1, self.heads, self.head_dim)
        k = self.k(x).view(-1, self.heads, self.head_dim)
        v = self.v(x).view(-1, self.heads, self.head_dim)

        # --- The Core Innovation ---
        # Project edge attributes to bias terms
        e_bias = self.edge_proj(edge_attr).unsqueeze(-1)

        # Calculate Raw Attention Scores (Q * K^T)
        score = (q[r] * k[c]).sum(dim=-1, keepdim=True) * self.scale

        # Add Edge Bias
        score = score + e_bias

        # Softmax normalization (using PyG sparse softmax)
        attn = pyg_softmax(score, r, num_nodes=x.size(0))
        attn = self.drop(attn)

        # Aggregate Values
        out = torch.zeros_like(v)
        out.index_add_(0, r, v[c] * attn) # Scatter add

        return self.out(out.view(-1, self.dim))
```

Deep Dive: 1. `edge_proj`: Transforms the edge embedding (e.g., "Jump") into a bias that affects how much attention is paid across that edge. 2. `q[r] *`

`k[c]`: Computes similarity only for connected nodes (Sparse Attention). 3. `index_add_`: Efficiently aggregates the weighted values back to the source nodes.

3.5. The Transformer Block (`GraphTransformerBlock`)

Encapsulates the Attention and FFN into a residual block.

```
class GraphTransformerBlock(nn.Module):
    def __init__(self, dim, heads, edge_dim=32, drop=0.1):
        super().__init__()
        self.norm1 = nn.LayerNorm(dim)
        self.attn = EdgeAugmentedAttention(dim, heads, edge_dim, drop)
        self.norm2 = nn.LayerNorm(dim)
        self.ff = nn.Sequential(
            nn.Linear(dim, dim*4), nn.GELU(), nn.Dropout(drop),
            nn.Linear(dim*4, dim), nn.Dropout(drop)
        )

    def forward(self, x, edge_index, edge_attr):
        # Pre-Norm Architecture
        # x = x + Attention(Norm(x))
        x = x + self.attn(self.norm1(x), edge_index, edge_attr)

        # x = x + FFN(Norm(x))
        x = x + self.ff(self.norm2(x))
        return x
```

3.6. The Full Model (`GNN_Deobfuscator`)

Assembles the components into the final architecture.

```
class GNN_Deobfuscator(nn.Module):
    def __init__(self, vocab_size, embed_dim=256, op_dim=32, layers=6, heads=8):
        super().__init__()

        # 1. Embeddings
        self.emb_mnem = nn.Embedding(vocab_size, embed_dim)
        self.emb_op = nn.Embedding(5, op_dim)
        self.emb_edge = nn.Embedding(4, 32)

        # 2. Feature Fusion
        self.fusion = nn.Linear(embed_dim + 2*op_dim, embed_dim)

        # 3. Positional Encoding
        self.register_buffer('pe', self._gen_pe(embed_dim))
```

```
        # 4. Encoder Stack
        self.layers = nn.ModuleList([
            GraphTransformerBlock(embed_dim, heads) for _ in range(layers)
        ])

        # 5. Classification Head
        self.head = nn.Linear(embed_dim, 1)

    def forward(self, batch):
        # Lookup Embeddings
        mnem_emb = self.emb_mnem(batch.x_mnem)
        op1_emb = self.emb_op(batch.x_op1)
        op2_emb = self.emb_op(batch.x_op2)

        # Fuse Features
        x = torch.cat([mnem_emb, op1_emb, op2_emb], dim=-1)
        x = self.fusion(x)

        # Add Positional Encoding
        pos = batch.pos.clamp(max=self.pe.size(0)-1)
        x = x + self.pe[pos]

        # Process Layers
        e_attr = self.emb_edge(batch.edge_attr)
        for layer in self.layers:
            x = layer(x, batch.edge_index, e_attr)

        # Final Prediction (Logits)
        return self.head(x).squeeze(-1)
```

3.7. Data Injection (`OLLVMInjector`)

This class simulates the adversary. It injects "Diamond" control flow structures (a common OLLVM obfuscation pattern) into the training data on-the-fly.

```
class OLLVMInjector:
    def gen_diamond_graph(self, context_buffer):
        # ...
        # 1. Create Predicate (CMP)
        # 2. Create Conditional Jump (JZ/JNZ)
        # 3. Create Fake Block (Junk Instructions)
        # 4. Wire them together in a diamond shape
        # ...
        return nodes, labels, edges
```

Logic: 1. **Predicate**: A comparison is generated (`CMP`). 2. **Divergence**: A conditional jump splits execution into two paths. 3. **Fake Path**: One path

contains randomly generated "junk" instructions (labeled `1.0`). 4. **Real Path**: The original code (labeled `0.0`). 5. **Convergence**: Both paths merge back, confusing static analysis tools.

3.8. Graph Parsing (`GraphParser`)

Converts raw assembly text into the PyTorch Geometric `Data` object format.

```
class GraphParser:
    def parse(self, path, max_nodes=2500):
        # ...
        # 1. Read file
        # 2. Regex parse lines into (Mnemonic, Op1, Op2)
        # 3. Randomly trigger OLLVMInjector to add junk
        # 4. Construct Edge Index (Sequential flow + Jumps)
        # 5. Return Data object
        # ...
```

---

4. Training Metrics Table

The following (simplified) table tracks the core metrics per epoch.

Epoch	Batch	Train Loss (Focal)	Val Loss (Focal)	Val F1 Score
1	End	0.0810	0.0569	0.8597
2	End	0.0578	0.0525	0.8680
3	End	0.0548	0.0519	0.8647
4	End	0.0530	0.0502	0.8737
5	End	0.0516	0.0496	0.8715
6	End	0.0505	0.0484	0.8778
7	End	0.0494	0.0478	0.8773
8	End	0.0486	0.0474	0.8759
9	End	0.0481	0.0473	0.8779
10	End	0.0476	0.0470	0.8780
11	End	0.0473	0.0469	0.8765
12	End	0.0471	0.0466	0.8846
13	End	0.0468	0.0461	0.8844
14	End	0.0467	0.0461	0.8838
15	End	0.0464	0.0460	0.8822
16	End	0.0463	0.0461	0.8820
17	End	0.0462	0.0457	0.8846
18	End	0.0461	0.0456	0.8870
19	End	0.0459	0.0455	0.8850
20	End	0.0458	0.0455	0.8886

Inference Results (50 test samples)

Evaluation on the held-out 50-sample test set:

Accuracy	F1 Score	Precision	Recall
0.8662	0.8865	0.9025	0.8711

Metric Definitions: - **Train/Val Loss**: The value of the Focal Loss function. Lower is better. - **F1 Score**: Harmonic mean of Precision and Recall. Critical for imbalanced datasets. - **Precision (Junk)**: $\frac{TP}{TP+FP}$. Of all instructions predicted as junk, how many were actually junk? - **Recall (Junk)**: $\frac{TP}{TP+FN}$. Of all actual junk instructions, how many did we find?

---

5. Architecture Description

the architecture can be visualized as follows:

1. **Input**: Sequence of Assembly Instructions (ASM File).
2. **Parsing & Graph Construction**:
   - Nodes: Instructions.
   - Edges: Control Flow (Next instruction, Jumps).
   - *Augmentation*: Synthetic Diamond Graphs injected.
3. **Embedding Layer**:
   - Mnemonic ID $\rightarrow$ Vector (256d).
   - Operand Types $\rightarrow$ Vector (32d).
   - Concatenate & Fuse $\rightarrow$ Node Vector (256d).
4. **Positional Encoding**: Add Sine/Cosine waves to Node Vectors.
5. **Graph Transformer Encoder (x6)**:
   - Input: Node Vectors + Edge Indices + Edge Types.
   - Layer 1-6: Apply Edge-Augmented Attention + FFN.
   - Output: Contextualized Node Vectors (256d).
6. **Classification Head**:
   - Linear Projection (256d $\rightarrow$ 1d).
   - Sigmoid Activation (Implicit in BCEWithLogits).
7. **Output**: Probability $P(y = 1|x)$ for each instruction (Probability of being Junk).

---

6. The Synthetic-to-Real Generalization Problem

A critical question in this architecture is the validity of training on synthetic obfuscation (via `OLLVMInjector`) to detect real-world obfuscation.

6.1. Mathematical Formulation of Distribution Shift

This problem is formally characterized as **Domain Adaptation** or **Out-of-Distribution (OOD) Generalization**.

Let $\mathcal{X}$ be the space of all possible control flow graphs and $\mathcal{Y} = \{0, 1\}^N$ be the label space (Real vs. Junk). We define two distributions: 1. **Source Distribution (Synthetic)**: $P_S(X, Y)$, generated by our `OLLVMInjector`. 2. **Target Distribution (Real-World)**: $P_T(X, Y)$, generated by unknown malware authors or commercial packers (e.g., VMProtect).

The Problem:

$$P_S(X, Y) \neq P_T(X, Y)$$

Specifically, we face **Covariate Shift**, where the marginal distribution of inputs changes ($P_S(X) \neq P_T(X)$), while the conditional probability of labels *might* remain similar ($P_S(Y|X) \approx P_T(Y|X)$) if the underlying definition of "junk code" (dead code that does not affect the program state) remains constant.

6.2. Theoretical Absence of Proof

There is currently no mathematical proof that a deep learning model trained on P_S will minimize the risk on P_T:

$$R_T(f) = \mathbb{E}_{(x,y)\sim P_T}[\mathcal{L}(f(x), y)]$$

Standard Statistical Learning Theory (e.g., PAC-learning) assumes that training and test data are drawn i.i.d. from the *same* distribution. When this assumption is violated, the error on the target domain is bounded by:

$$R_T(f) \leq R_S(f) + \frac{1}{2} d_{\mathcal{H}\Delta\mathcal{H}}(P_S, P_T) + \lambda$$

Where $d_{\mathcal{H}\Delta\mathcal{H}}$ is the divergence between the two domains. If the synthetic obfuscation is too simple (low divergence), the model learns specific artifacts of the generator. If the real-world obfuscation is significantly more complex (high divergence), the bound becomes loose, and performance guarantees vanish.

6.3. Empirical Justification: The Structural Invariant Hypothesis

Despite the lack of formal proof, this approach is widely adopted in research (e.g., *Debin*, *Neural Reverse Engineering*) based on the **Structural Invariant Hypothesis**.

Hypothesis: While the *surface syntax* of obfuscation varies (e.g., different registers, different junk instructions), the *topological structure* of the Control Flow Graph (CFG) induced by obfuscation techniques (like Control Flow Flattening) shares common invariants.

1. **High Cyclomatic Complexity**: Obfuscated graphs tend to have artificially high connectivity.

2. **Unreachable Code**: Junk blocks often have specific connectivity patterns (e.g., the "Diamond" shape simulated in `OLLVMInjector`).
3. **Dispatcher Nodes**: Flattening introduces central dispatcher nodes with high in-degree.

The GNN is designed to learn these **topological invariants** rather than specific instruction sequences. By using **Edge-Augmented Attention**, the model focuses on the *geometry* of the code ($P(Y|\text{Graph Structure})$) rather than the *text* of the code ($P(Y|\text{Mnemonics})$), which is more robust to domain shift.

6.4. The Role of the GNN: Neural Heuristic vs. Formal Verifier

Given the mathematical uncertainty, the GNN in this system should be viewed as a **Neural Heuristic** (a proposal distribution) rather than a formal verifier.

$$\text{Deobfuscation Process} = \text{Verify}(\text{Propose}_{GNN}(X))$$

1. **Propose**: The GNN estimates $\hat{Y} = f_\theta(X)$. It identifies *candidate* junk instructions with high probability.
2. **Verify**: A secondary system (e.g., Symbolic Execution or Reinforcement Learning) verifies if removing these candidates preserves the program's semantics.

This hybrid approach mitigates the risk of the unproven generalization. The GNN reduces the search space from exponential to manageable, and the verifier ensures correctness, bypassing the need for a purely mathematical proof of GNN generalization.

---

7. Conclusion

The `GNN_Deobfuscator` represents a specialized application of Graph Representation Learning to the domain of binary security. By treating assembly code as a graph with sequential properties, and by explicitly modeling the control flow edges via the attention mechanism, the model is theoretically capable of distinguishing between functional code and obfuscated "dead" code structures. The use of Focal Loss ensures that the model remains sensitive to the sparse "junk" signals amidst the noise of legitimate instructions.